



FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGIES
SUMMER 2026
FINAL EXAMINATION

COURSE TITLE:	Large System Environment
COURSE CODE:	SEN4121
INSTRUCTOR:	Engr. Tanwi Nkiamboh
DURATION:	2 weeks
GROUP SIZE:	4 Software Engineering students per group

INSTRUCTIONS

- Each group will deliver a 5-minute live-system demonstration followed by a 5-minute individual defense Q&A per member — every member must be able to justify any part of the codebase.
- Your SRS + technical report should not exceed 20 pages, excluding appendices and diagrams.
- Submit a public/private Git repository link with a complete commit history — squashed single-commit submissions will be penalized under the DevOps criterion.
- Each member must submit a 1-page individual contribution statement signed by all group members; unequal contribution may result in individualized mark adjustments.
- A signed academic integrity and originality declaration must accompany the submission. Reused or AI-generated code must be disclosed and explained during the defense.
- Late submissions are penalized 5 marks per day, up to a maximum of 3 days, after which the submission is not graded.

PROJECT

ERP System Development for Academic, Marketing & Finance, and Administration & Human Resource Modules — Microservices Edition

Objective

Design, document, and implement a production-grade, multi-tenant ERP system composed of independently deployable microservices covering three core domains: Academic, Marketing & Finance, and Administration & Human Resources. Groups must produce a Software Requirements Specification (SRS) aligned with ISO/IEC/IEEE 29148, a complete UML diagram suite, and a working system deployed via containers with an automated CI/CD pipeline. Emphasis is placed on architectural soundness, security, and defensible engineering decisions — not merely feature completeness.

Mandatory Architectural Constraints

- The system must be decomposed into at least three independently deployable microservices (one per module), each with its own database schema (database-per-service pattern).
- An API Gateway must front all services and handle routing, authentication delegation, and rate limiting.
- Inter-service communication for at least one cross-module workflow (e.g., a new student enrollment triggering a finance invoice) must use asynchronous messaging (RabbitMQ, Kafka, or equivalent) — direct synchronous calls alone are insufficient.

- All services must be containerized with Docker and orchestrated locally via docker-compose (Kubernetes manifests are optional bonus work).
- A CI/CD pipeline (GitHub Actions, GitLab CI, or equivalent) must automatically run the test suite and build container images on every push to the main branch.

PROJECT SCOPE

A. General Features (Common to All Modules) — 34 Marks

1. User Authentication & Authorization (6 Marks):

- Role-based access control with at least four roles (e.g., Super Admin, Admin, Staff, Student), enforced at both API and UI layers.
- Secure login using bcrypt or Argon2 password hashing (never plain MD5/SHA1); JSON Web Tokens (JWT) with short-lived access tokens and refresh-token rotation.
- Session management, forced logout on token expiry, and account lockout after repeated failed login attempts.
- Bonus: optional multi-factor authentication (email/SMS OTP) for admin accounts.

2. Database Management (6 Marks):

- Relational database (MySQL or PostgreSQL) normalized to at least 3NF, with a documented ERD justifying design decisions.
- Indexing strategy for high-read tables and evidence of query optimization (e.g., EXPLAIN plans before/after).
- Automated backup and point-in-time recovery procedure, tested and documented with screenshots or logs.
- Transaction management (ACID) demonstrated on at least one multi-step operation (e.g., payment + ledger update).

3. Microservices Architecture & API Integration (7 Marks):

- RESTful APIs documented with OpenAPI/Swagger, including request/response schemas and error codes.
- API versioning strategy (e.g., /api/v1/) and rate limiting at the gateway.
- At least one asynchronous, message-broker-driven workflow spanning two or more services.
- Service discovery or a documented static service registry.

4. Non-Functional Requirements (5 Marks):

- Load-testing results (e.g., using Apache JMeter or k6) showing response times under a defined concurrent-user target.
- Mitigation evidence for at least five OWASP Top 10 risks (e.g., SQL injection, XSS, broken auth, CSRF, insecure deserialization).
- Centralized logging and basic monitoring/health-check endpoints for each service.
- Documented horizontal-scaling strategy, even if only demonstrated conceptually.

5. Testing & Quality Assurance (4 Marks):

- Unit and integration test suites with a minimum of 70% code coverage on core business logic.
- A written test plan mapping test cases to functional requirements.
- Evidence of bug tracking (issue tracker board, e.g., GitHub Issues/Trello) with resolved and open items.

6. DevOps: Containerization, CI/CD & Deployment (6 Marks):

- Dockerfile per service and a working docker-compose.yml that brings up the full system with one command.
- CI pipeline configuration that runs automated tests and builds images on every commit to main.
- Deployment guide sufficient for a third party to run the system from a clean machine.

B. Module-Specific Requirements — 24 Marks

7. Academic Module (8 Marks):

- Functionalities: course/program CRUD, prerequisite enforcement, student performance tracking (grades, attendance), examination scheduling with conflict detection, results publication with grade-appeal workflow.
- Deliverables: dynamic dashboards for students and instructors, automated grade-transcript and attendance-summary reports (PDF export), and a simple at-risk-student flag based on a documented rule or basic statistical threshold (e.g., attendance below 75% or two consecutive failing grades).

8. Marketing & Finance Module (8 Marks):

- Functionalities: tuition invoice generation and payment tracking, expense tracking and financial reporting, marketing campaign tracking (leads, conversions, ROI).
- Deliverables: fee payment portal with digital receipts and integration with a mobile-money sandbox/simulated gateway (MTN MoMo or Orange Money API, or a mocked equivalent if sandbox access is unavailable), analytics dashboard for campaign and financial metrics, and automated monthly financial summary reports in FCFA.

9. Administration & Human Resource Module (8 Marks):

- Functionalities: employee management (recruitment, payroll, attendance), leave and performance tracking, asset/inventory management.
- Deliverables: payroll computation that accounts for standard Cameroonian statutory deductions (CNPS contribution and PAYE tax bracket, using published rates), QR-code or simulated biometric attendance check-in, HR dashboard with leave status and performance analytics, and automated notifications for critical tasks such as leave approval.

C. Documentation & Diagram Suite — 7 Marks

- A Software Requirements Specification aligned with ISO/IEC/IEEE 29148, including functional and non-functional requirements, assumptions, and constraints.
- A complete UML diagram suite: use-case diagram, class diagram, at least two sequence diagrams (one covering the cross-service asynchronous workflow), an ERD, and a deployment diagram showing containers and the message broker.
- Diagrams must be consistent with the delivered implementation — inconsistencies between documentation and running code will be penalized.

D. Presentation, Live Demo & Defense — 5 Marks

- A live, unscripted demonstration of the running containerized system (not slides of screenshots).
- Individual defense questioning: each member must independently explain and, if asked, modify a small piece of their assigned component live.
- Marks are awarded per individual based on depth of understanding demonstrated, not only on team output.

MARK ALLOCATION SUMMARY

Assessment Component	Marks
1. User Authentication & Authorization (RBAC, JWT/OAuth2, MFA)	6
2. Database Design & Management (normalization, indexing, backup/recovery, replication)	6
3. Microservices Architecture & API Integration (gateway, versioning, message broker)	7
4. Non-Functional Requirements (performance, scalability, OWASP-aligned security, logging)	5
5. Testing & Quality Assurance (unit/integration tests, coverage, bug tracker)	4
6. DevOps: Containerization, CI/CD & Deployment	6

7. Academic Module (functionalities + deliverables + AI-assisted analytics)	8
8. Marketing & Finance Module (functionalities + deliverables + mobile-money integration)	8
9. Administration & HR Module (functionalities + deliverables + biometric/QR attendance)	8
10. SRS Documentation & UML Diagram Suite	7
11. Presentation, Live Demo, Defense Q&A & Individual Contribution	5
TOTAL	70

DELIVERABLES CHECKLIST

- SRS document (PDF), UML diagram suite, and ERD.
- Complete source code in a Git repository with commit history (no single-commit dumps).
- Dockerfiles, docker-compose.yml, and CI/CD pipeline configuration.
- Deployment/setup guide and API documentation (Swagger/OpenAPI export).
- Test plan, test reports, and coverage report.
- Individual contribution statement and signed academic integrity declaration.
- Short recorded video (max 5 minutes) as a fallback demo in case of live-demo technical failure.

ACADEMIC INTEGRITY

Any code, diagram, or document reused from external sources, classmates from other groups, or generated with AI assistance must be explicitly cited in the report and disclosed at the start of the defense. Undisclosed reuse discovered during grading will be treated as an academic integrity violation under Faculty policy and may result in a zero for the affected component or the entire project.

ALL THE BEST